DBMS PROJECT-BASED LEARNING • REVIEW 1

Car Rental Reservation & Vehicle Return Management System

Conceptual Design, ER Model & Initial Relational Schema



Presented by: Koneri Mithul | Roll Number(s) -25WU0102129 | Class & Section-AIML Panthers

Guide: Dr. Kiran Mayee Adavala | Department of Computer Science

OVERVIEW

What We'll Cover

1

Problem Statement

Why manual car-rental tracking fails

2

Objectives & Scope

What the system will and won't do

3

Users & Requirements

Who uses the system, and how

4

ER Model

11 entities and how they relate

5

Business Rules

5 rules the database must enforce

6

Initial Schema

Tables, keys, and what's next

THE PROBLEM

Manual Tracking Breaks Down at Scale

Car-rental agencies operating multiple branches currently track vehicle availability, customer records, bookings, and billing manually or across disconnected spreadsheets. As fleet size and branch count grow, this approach fails in predictable ways.

- No single source of truth for which vehicle is free, where, and when
- Reservations aren't checked against existing bookings before confirming
- No structured record of vehicle condition at pickup vs. return
- Renting to customers with an expired driving licence goes unnoticed

Double-booking

Same vehicle promised to two customers

Billing disputes

No proof of pre-existing damage

Revenue leakage

Missed late fees & extra-km charges

GOALS

Objectives & Scope

Objective

Design a normalized relational database, enforced with integrity constraints, and a Java/Python front-end that gives branch staff one reliable system for availability, reservations, pickup/return inspections, billing, and reporting.

In Scope

- ✓ Multi-branch fleet & category-based pricing
- ✓ Reservation → rental conversion workflow
- ✓ Pickup / return condition inspection
- ✓ Damage, invoicing & payment tracking
- ✓ Fleet, revenue & utilization reporting

Out of Scope (v1)

- ✗ Online payment gateway integration
- ✗ GPS / real-time vehicle tracking
- ✗ Dynamic, demand-based pricing engine
- ✗ Mobile app front-end

STAKEHOLDERS

Users & What They Need



Branch Staff

Search availability, create reservations, issue & receive vehicles, log inspections, bill customers



Branch Manager

Monitor fleet utilization, review revenue by branch, track overdue rentals & damages

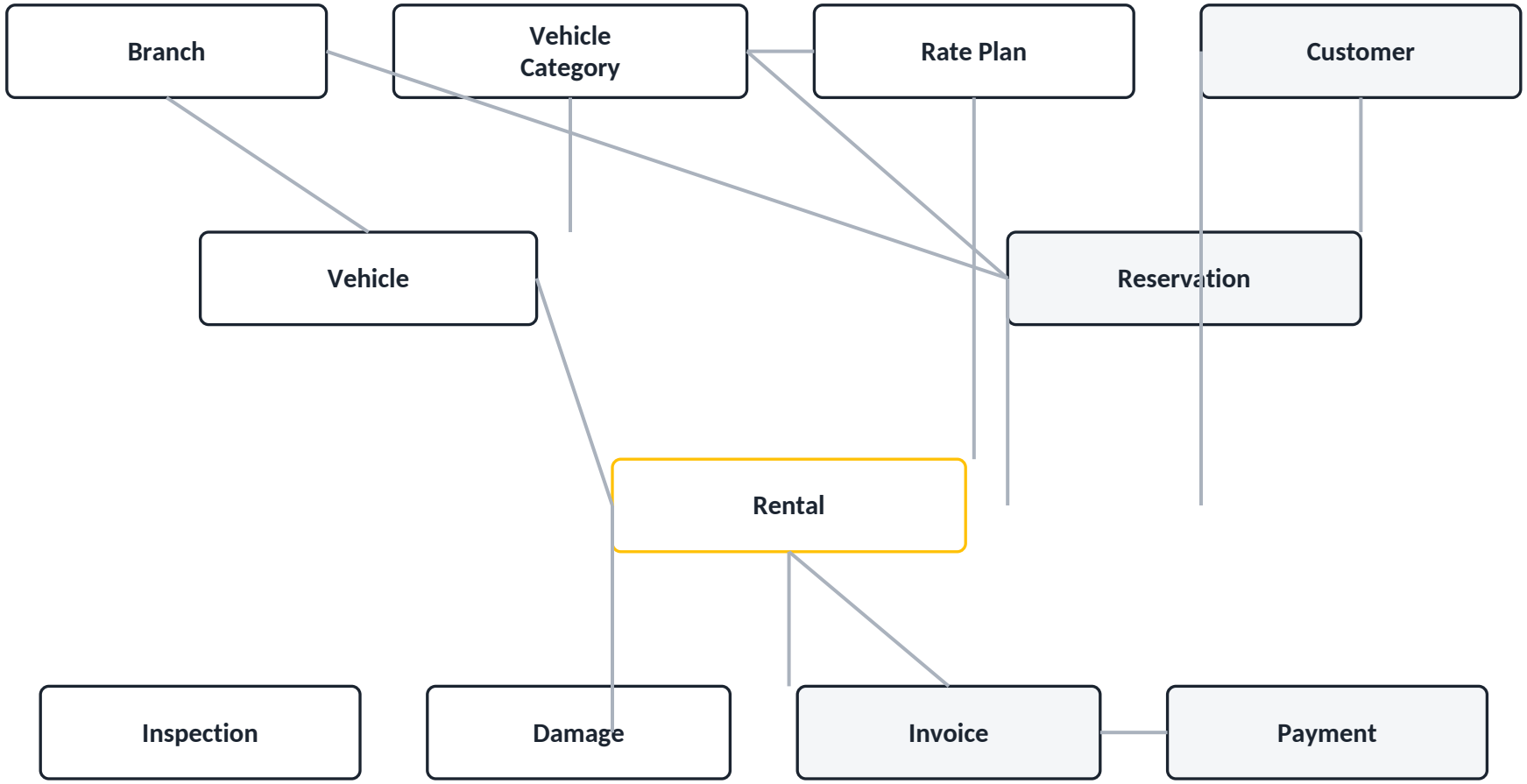


Customer

Check vehicle availability by category and branch, book in advance, view charges

CONCEPTUAL DESIGN

Entity-Relationship Model



11 entities • lines = PK→FK relationships (1:M unless the Rental box is involved, which is 1:0..1 from Reservation and 1:1 to Invoice)

CONCEPTUAL DESIGN

Entities at a Glance

Entity	Primary Key	Role
Branch	branch_id	Rental office location
VehicleCategory	category_id	Economy / Sedan / SUV / Luxury
Vehicle	vehicle_id	Individual fleet vehicle
Customer	customer_id	Renter with licence details
RatePlan	rate_plan_id	Pricing per category
Reservation	reservation_id	Advance booking, pre-vehicle-assignment
Rental	rental_id	Actual transaction — vehicle issued
Inspection	inspection_id	Condition at pickup and at return
Damage	damage_id	Damage recorded during a rental
Invoice	invoice_id	Final bill for one rental
Payment	payment_id	Payment(s) against an invoice

DATA INTEGRITY

Business Rules We Must Enforce

No overlapping rentals	Same vehicle can't be rented to two customers on overlapping dates	**Trigger (cross-row check on INSERT)**
Valid driving licence	Customer's licence must not be expired at pickup	**Trigger (checks Customer.license_expiry)**
Return after pickup	actual_return_datetime must be later than pickup_datetime	**CHECK constraint**
Valid odometer / fuel readings	Return odometer ≥ pickup odometer; fuel 0–100%	**CHECK + Trigger**
Non-negative charges	All amounts on Invoice / Payment must be ≥ 0	**CHECK constraint**

FROM ER TO RELATIONS

Initial Relational Schema

```
Branch(branch_id PK, branch_name, city, address, phone)

VehicleCategory(category_id PK, category_name, description)

Vehicle(vehicle_id PK, registration_no UK, category_id FK, branch_id FK, ..., status)

Customer(customer_id PK, full_name, phone UK, license_no UK, license_expiry, ...)

RatePlan(rate_plan_id PK, category_id FK, daily_rate, extra_km_rate, late_fee_per_hour)

Reservation(reservation_id PK, customer_id FK, category_id FK, pickup_branch_id FK, return_branch_id FK, ...)

Rental(rental_id PK, reservation_id FK, vehicle_id FK, customer_id FK, rate_plan_id FK, ...)

Inspection(inspection_id PK, rental_id FK, inspection_type, odometer_reading, fuel_level_percent, ...)

Damage(damage_id PK, rental_id FK, inspection_id FK, description, repair_cost, charged_amount)

Invoice(invoice_id PK, rental_id FK UK, base_charge, extra_km_charge, late_fee, total_amount)

Payment(payment_id PK, invoice_id FK, amount, payment_mode, transaction_ref)
```

STATUS

Review 1 Deliverables — Complete

- ✓ Problem identification, scope, objectives, users and functional requirements
- ✓ Conceptual design / ER diagram covering all 11 principal entities
- ✓ Initial relational schema with proposed tables, primary keys and relationships

Next: Review 2 — normalize to 3NF, write DDL with constraints, load sample data, and build reporting queries/views.



Thank You

Questions & Discussion